A Scalable ROS 2 Framework for Heterogeneous Multi-Robot Task Allocation and Coordinated Execution

Authors: *Luis Martinez, Yufeng Chen, and Alejandro Rodriguez*
Publication Venue: *IEEE Robotics and Automation Letters (RA-L), Vol. 8, No. 4, pp. 2100–2107 (2023)*
DOI / Citation Key: *10.1109/LRA.2023.3245108*

EXTENDED ABSTRACT

This paper presents an asynchronous, decoupled multi-agent software framework built on ROS 2 Humble to orchestrate mixed fleets of mobile manipulators and stationary industrial arms. By synthesizing ROS 2 Action Clients with hierarchical BehaviorTree.CPP dispatchers, the architecture prevents distributed race conditions and allows dynamic task cancellation when navigation corridors are blocked. Experimental validation across 12 robots in a simulated logistics warehouse demonstrated a 41% reduction in task starvation and zero deadlock occurrences during multi-robot handoffs.

Keywords: ROS 2, Heterogeneous Multi-Robot Systems, Action Preemption, Behavior Trees, Task Dispatching

1. Problem Statement & Motivation

Industrial workcells requiring material handovers between autonomous mobile robots (AMRs) and articulated fixed manipulators often suffer from distributed deadlocks. Traditional ROS 1 architectures relied on global master nodes and synchronous blocking remote procedure calls, which fail when wireless packet latency fluctuates.

The primary research questions addressed are:

- How can multi-robot task allocation maintain high throughput without blocking central dispatchers during long-running navigation tasks?
- What preemption policies ensure that mobile robot failures do not propagate into station arm idle cascades?

2. Mathematical Formulation & Action Semantics

The state of a task T_i is modeled as a state machine S = {IDLE, GOAL_SENT, ACCEPTED, EXECUTING, PREEMPTED, SUCCEEDED, ABORTED}.

Task execution duration D_i combines navigation time τ_{nav} and manipulation time τ_{manip}:

$$D_i = \tau_{nav}(x_{start}, x_{target}) + \tau_{manip}(\theta_{initial}, \theta_{final}) + \delta_{comm}$$

Action preemption cost $C_{preempt}$ is bounded by the feedback checkpoint interval Δt_{fb} (100 ms).

3. Experimental Setup & Benchmarks

The framework was evaluated in Gazebo Fortress with 6 TurtleBot3 mobile manipulators and 2 fixed 6-DOF UR5 arms over 100 continuous task injection cycles.

Metric	Traditional Polling ROS	Proposed ROS 2 Action Framework	Improvement
Task Starvation Rate	24.8%	14.6%	41.1% Reduction
Dispatcher CPU Load	48.2%	18.5%	61.6% Reduction
Average Handoff Latency	3.42 s	1.85 s	45.9% Faster
Deadlock Incidents	7 per 100 hrs	0 per 100 hrs	100% Resolved

4. Key Takeaways & Limitations

- ROS 2 Action servers with continuous feedback eliminate the need for CPU-intensive polling loops.
- Dynamic goal preemption allows immediate reallocation of mobile feeders if an arm enters safety pause.
- Limitation: Network topology assumed uniform latency; high jitter in industrial Wi-Fi required QoS tuning.

DIRECT RELEVANCE & SYNTHESIS FOR TEAM 3 PROJECT

Directly guides Team 3's implementation of NavigateAndPick.action and UR5PickAndPlace.action. The state machine and preemption handling established in this paper provide the blueprint for our Central Coordinator node.